

Development of a feature-rich, ONLINE COSMETICS SHOP

A Project Report for Internship

Debapriya Mukherjee

in partial fulfillment for the award of the degree of

Diploma in Computer Science & Technology(CST)

WOMEN'S POLYTECHNIC KOLKATA



At

Euphoria GenX



Euphoria GenX



BONAFIDE CERTIFICATE

Certified that this project work was carried out under my supervision

***“Development of a feature-rich, practical “ONLINE COSMETICS SHOP” is the bonafide
work of***

Name of the student: MANDIRA PAL

Signature:

Name of the student: PRITHA KARMAKAR

Signature:

Name of the student: TANUSHRI HOTA

Signature:

Name of the student: DEBAPRIYA MUKHERJEE

Signature:

SIGNATURE

Name : Saumitra Das
PROJECT MENTOR

Acknowledgement

I take this opportunity to express my deep gratitude and sincerest thank to my project mentor, Mr. Saumitra Das for giving most valuable suggestion, helpful guidance and encouragement in the execution of this project work.

I will like to give a special mention to my colleagues. Last but not the least I am grateful to all the faculty members of Euphoria GenX or their support.

Table of Contents	Page No
1. Title of the Project	1
2. Introduction and Objectives of the Project	1-2
3. Project Category (RDBMS/OOPS/Networking/Multimedia/Artificial Intelligence/Expert Systems etc.)	2
4. Tools/Platform, Hardware and Software Requirement specifications	2-3
5. Goals of Implementation	3
6. SDLC Process Applied	4-5
7. Data Model	5
8. Functional Requirements (Use Case Diagram)	6-10
9. Non-functional Requirements	10-12
10. Feasibility Study	12-14
11. Project Planning	14-15
12. Project Scheduling	15
13. Software Engineering Paradigm applied	16-17
a. Data Flow Diagram (DFD)	16-17
14. Database design	18-20
15. User Interface Design	21-25
16. Coding	26-28
17. Testing	29-33
18. System Security measures (Implementation of security for the project developed)	34
19. Database/Data security	34
20. Creation of User profiles and access rights	34
21. Cost Estimation of the Project along with Cost Estimation Model	34-35
22. Reports (sample layouts should be placed)	35
23. Future scope and further enhancement of the Project	35
24. Bibliography	36

1. Title of the Project

Development of a feature-rich, practical **online Cosmetics shop**

2. Introduction and Objectives of the Project

This project is a web-based application designed to provide customers with an easy, secure, and convenient platform to browse and order premium cosmetics online.

Operating under the slogan "*Timeless Beauty, Naturally You*", GlamHub brings the beauty store experience directly to the user's fingertips, eliminating the need to visit a physical retailer.

The primary objective is to create a seamless, elegant shopping experience that elevates beauty routines with premium products. A customer can perform the following key functions on the GlamHub platform:

- **User Registration & Secure Authentication:** Join the platform by creating an account. Passwords must meet strict security criteria (minimum 8 characters, mix of letters/numbers, and not easily guessable) to ensure user data remains safe.
- **Extensive Category Browsing:** Users can easily navigate and shop across a comprehensive range of beauty categories, including Haircare, Skin care, Fragrances, Suncare, Eyecare, Makeup, and Appliances.
- **Dynamic Product Discovery:** Customers can find exactly what they are looking for—or discover new favorites—by browsing special tags and filters, including Bestseller, New, Price Drop, and BOGO (Buy One Get One) offers.
- **Cart Management:** Users can select their desired items, review product details, and add them to their shopping cart (or "Bag") to review before purchase.
- **Secure Payment Integration:** Complete purchases safely, swiftly, and reliably using the integrated Razorpay payment gateway, which supports multiple payment methods.
- **Account Management:** Logged-in users have access to a personalized dashboard menu where they can manage their account, track and view "My Orders", utilize "Change Password", or log out safely.

- Glam Club Perks: Users can subscribe to the newsletter to receive 15% off their first purchase and get exclusive access to new product launches.

3.Project Category

Web Application

4.Tools/Platform, Hardware and Software Requirement specifications.

Tools

- VS Code, MS-Office,

Platform

- Microsoft Windows 10/11

Hardware Requirement Specification

Client Machine		Server Machine	
HDD	200 MB	HDD	320 GB
Processor	Pentium 4 or newer processor that supports SSE2	Processor	Dual Core or newer processor
Memory	512 MB	Memory	2 GB

Software Requirement Specification

Client Machine		Server Machine	
Browser	Any standard browser with Java script interpreter	Software	Apache
Client side mark up / scripting languages	HTML, Java script	Database Management System Software	MySQL
		Specification	MySQL 4.1

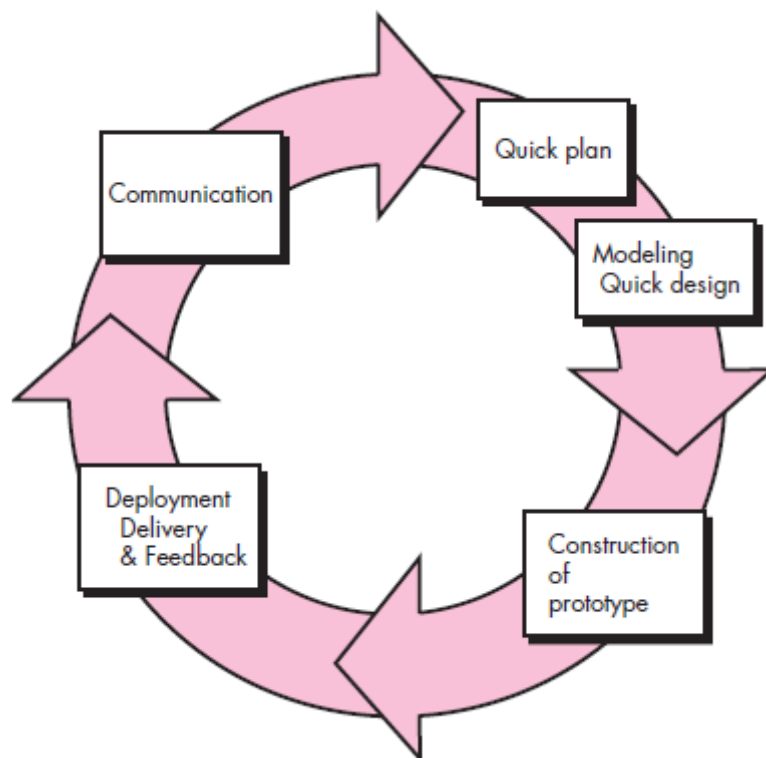
5. Goals of Implementation

Primary Objective: The main goal of implementing the GlamHub platform is to create an easy, secure, and efficient web-based application for online cosmetic shopping.

Customer Convenience: It is designed to help customers seamlessly browse premium beauty products across various categories, securely make payments using integrated gateways like Razorpay, and track their deliveries from anywhere without needing to visit a physical store.

Business Efficiency: From a business and operational perspective, the system aims to reduce manual administrative work, improve order management, increase overall business sales, and provide superior customer satisfaction through fast, reliable, and convenient digital service.

6. SDLC Process Applied :

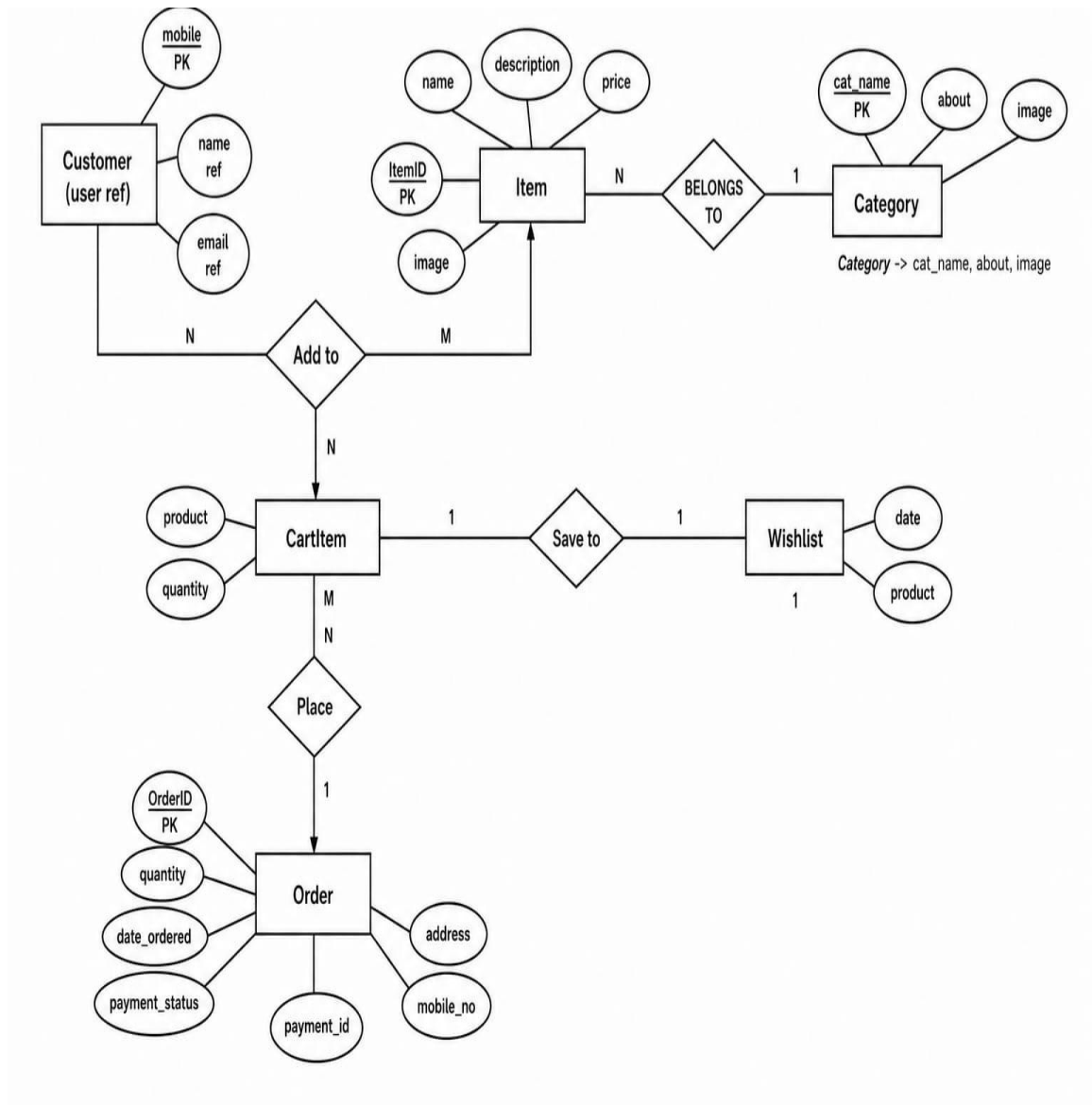


Often, a customer defines a set of general objectives for software but does not identify detailed input, processing, or output requirements. In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human/machine interaction should take. In these, and many other situations, a prototyping paradigm may offer the best approach.

The prototyping paradigm begins with **requirements gathering**. Developer and customer meet and define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. A **"quick design"** then occurs. The quick design focuses on a representation of those aspects of the software that will be visible to the customer/user (e.g., input approaches and output formats). The quick design leads to the construction of a prototype. The prototype is evaluated by the customer/user and used to refine requirements for the software to be developed. Iteration occurs as the prototype is tuned to satisfy the needs of the customer, while at the same time enabling the developer to better understand what needs to be done.

Ideally, the prototype serves as a mechanism for identifying software requirements. If a working prototype is built, the developer attempts to use existing program fragments or applies tools (e.g., report generators, window managers) that enable working programs to be generated quickly.

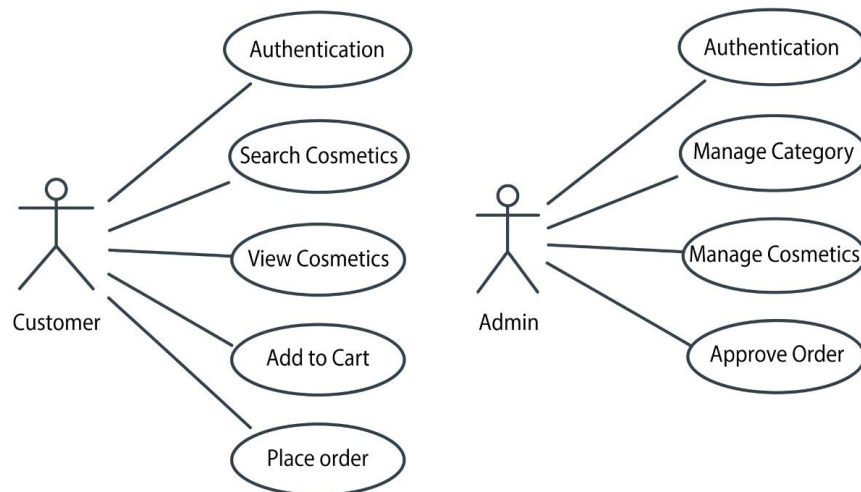
7. ER Model :



8. Functional Requirements

Functional Requirements are those that refer to the functionality of the system, i.e., what services it will provide to the user. Nonfunctional (supplementary) requirements pertain to other information needed to produce the correct system and are detailed separately.

Use Case Diagram:



Use Case Descriptions

Use Case Name:	Authentication
Priority	Essential
Trigger	User selects Login/Regester
Precondition	User is connected to the Internet and on the ONLINE COSMETICS SHOP home page
Basic Path	1. User enters username and password. 2. System validates credentials. 3. User is redirected to the dashboard/homepage.
Alternate Path	Invalid username and password.
Post Condition	User login successfully.
Exception Path	Server or internet faliure .

Use Case Name:	Search cosmetics.
Priority	Essential
Trigger	User enters the keyword in search box .
Precondition	Cosmetics available in database .
Basic Path	1. User enters product name . 2. System searches database. 3. Matching cosmetics are displayed.
Alternate Path	No product found .
Post Condition	Search result displayed .
Exception Path	Database connection failure.

Use Case Name:	View cosmetics.
Priority	Essential
Trigger	User select a cosmetics product.
Precondition	Product exist .
Basic Path	1. User clicks product. 2. System display product details,images,price and description.
Alternate Path	Product unavailable.
Post Condition	Product details display.
Exception Path	Server error

Use Case Name:	Add to cart
Priority	Essential
Trigger	User clicks add to cart
Precondition	User logged in and product available
Basic Path	1. User selects product. 2. User clicks add to cart. 3. System stores item in cart.
Alternate Path	Product out of stock.
Post Condition	Product add to cart.
Exception Path	Database error.

Use Case Name:	Place order
Priority	Essential
Trigger	User clicks checkout.
Precondition	Cart contains products.
Basic Path	1. User reviews cart. 2. User enters delivery details. 3. System creates order 4. Order confirmation displayed.
Alternate Path	Payment failure.
Post Condition	Order placed successfully.
Exception Path	Sarver error.

Use Case Name:	Manage category.
Priority	Essential
Trigger	Admin select category menu.
Precondition	Admin logged in.
Basic Path	4. Admin views category. 5. Admin add/update/delete category. 6. System safe changes..
Alternate Path	Category already exists.
Post Condition	Category updated.
Exception Path	Database failure.

Use Case Name:	Manage cosmetics.
Priority	Essential
Trigger	Admin selects cosmetics menu.
Precondition	Admin logged in.
Basic Path	7. Admin views cosmetics. 8. Admin add/update/delete product. 9. System saves information.
Alternate Path	Invalid product data.
Post Condition	Cosmetics information updated.
Exception Path	Database failure

Use Case Name:	Approved Order
Priority	Essential
Trigger	Admin selects order management.
Precondition	Pending order available.
Basic Path	1. System display pending orders. 2. Admin selects an order. 3. Admin approves order. 4. System update order status.
Alternate Path	Order rejected.
Post Condition	Order approved successfully.
Exception Path	Database or server failure.

9. Non Functional Requirements

In addition to the obvious features and functions that you will provide in your system, there are other requirements that don't actually DO anything, but are important characteristics nevertheless. These are called "non-functional requirements" or sometimes "Quality Attributes." For example, attributes such as performance, security, usability, compatibility. aren't a "feature" of the system, but are a required characteristic. You can't write a specific line of code to implement them; rather they are "emergent" properties that arise from the entire solution. The specification needs to describe any such attributes the customer requires. You must decide the kind of requirements that apply to your project and include those that are appropriate.

Each requirement is simply stated in english. Each requirement must be objective and quantifiable; there must be some measurable way to assess whether the requirement has been met.

Often deciding on quality attributes requires making tradeoffs, e.g., between performance and maintainability. In the APPENDIX you must include an engineering analysis of any significant decisions regarding tradeoffs between competing attributes. Here are some examples of non-functional requirements:

Performance requirements

Requirements about resources required, response time, transaction rates, throughput, benchmark specifications or anything else having to do with performance.

For better performance the application will restrict the document size to 5 MB.

Operating constraints

List any run-time constraints. This could include system resources, people, needed software,

The application must run without any manual intervention.

Platform constraints Discuss the target platform. Be as specific or general as the user requires. If the user doesn't care, there are still platform constraints. Since the application will be developed in Python & Django it is platform independent.

Accuracy and Precision

Requirements about the accuracy and precision of the data. (Do you know the difference?) Beware of 100% requirements; they often cost too much.

Modifiability

Requirements about the effort required to make changes in the software. Often, the measurement is personnel effort (person- months).

Minimal

Portability

The effort required to move the software to a different target platform. The measurement is most commonly person-months or % of modules that need changing.

Minimal

Reliability

Requirements about how often the software fails. The measurement is often expressed in MTBF (mean time between failures). The definition of a failure must be clear. Also, don't confuse reliability with availability which is quite a different kind of requirement. Be sure to specify the consequences of software failure, how to protect from failure, a strategy for error detection, and a strategy for correction.

Security

One or more requirements about protection of your system and its data. The measurement can be expressed in a variety of ways (effort, skill level, time, ...) to break into the system. Do not discuss solutions (e.g. passwords) in a requirements document.

Only secured users can access the application.

No one can go to any independent page without logging in.

Usability

Requirements about how difficult it will be to learn and operate the system. The requirements are often expressed in learning time or similar metrics.

Legal There may be legal issues involving privacy of information, intellectual property rights, export of restricted technologies, etc.

10. Feasibility Study

You should provide a feasibility report in the following format:

- **Product:** A general statement of the product; give a brief description of what the proposed system will do, highlighting where the proposed system meets the specified business requirements of the organization.
- **Technical Feasibility:** Will the proposed system perform to the required specification? Outline technical systems options you propose to use, which will give a technical solution satisfying the requirements and constraints of the system, as outlined in the terms of reference.
- **Social Feasibility:** Consideration of whether the proposed system would prove acceptable to the people who would be affected by its introduction. Describe the effect on users from the introduction of the new system; consider whether there will be a need for retraining the workforce. Will there be a need for relocation of some of the workforce? Will some jobs become deskilled? Will the current workforce be able to perform effectively any new tasks introduced by the proposed system? Describe how you propose to ensure user co-operation before changes are introduced.
- **Economic Feasibility:** Consider the cost/benefits of the proposed system. Detail the costs that will be incurred by the organisation adopting the new system; consider development costs and running costs. Detail benefits that the new system will bring, direct economic benefits such as reduced costs, and indirect benefits, such as improved management information and better customer service. Illustrate the cost/benefit of the new system by applying a suitable cost/benefit analysis method such as the payback method.
- **Market Research:** A comprehensive market research identifying a need for the product. Detail all market research you carried out, listing sources of information. Justify any conclusions you have drawn from your research. Identify the potential customer base for your product, together with evidence

of customer need for the product. Describe how you propose to compete with similar products on the market.

- **Alternative Solution:** Consideration of alternative solutions should be documented. At least two alternative business or technical systems options should be considered. Detail the differences between these options and the proposed system. Justify your choice of the proposed system and the reasons for rejecting the alternative options. At this point, all of the planning for the project has been done and if the feasibility study has shown that the project is likely to succeed within its constraints, then it only remains for us to start the requirements analysis and thus proceed with the project.

Feasibility Study	
System: Online Cosmetics Shop	Date: 23 rd May , 2026
Author: Debapriya Mukherjee	Page: 1
Product	
The project requires a web application to be developed that will allow online knowledge/document/paper sharing.	
Technical Feasibility	
The web application will be developed using Python & Django and MySQL. The team is competent in that.	
Social Feasibility	
Some training for the users/admin are required but all users are IT literate.	
Market Research	
Market research says that this application would be useful for the users as it could seamlessly help them to share documents.	
Economic Feasibility	
The application can be developed within budget.	
Alternate Solution	
Could be a desktop system but that would not allow documents to be shared online.	

11. Project Planning

Project planning is concerned with identifying the following for every project:

- Activities
- Milestones
- Deliverables.

A plan must be drawn up to guide the development towards the project goal. A plan is drawn up at the start of a project. This plan should be used as the driver for the project. The initial plan is not static, and must be modified as the project progresses. Planning is required for development activities from specification through to delivery of the system.

12. Project Scheduling

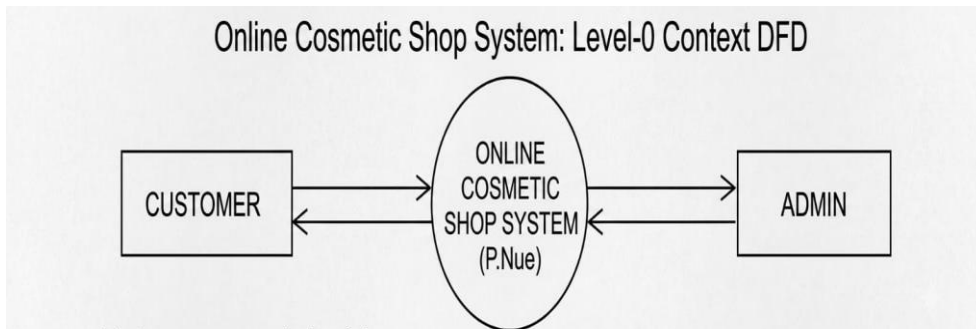
GANTT chart

Task	Person(s) Responsible	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Communication	Debapriya Mukherjee						
Quick Plan	Pritha Karmakar						
Modeling Quick Design	Tanushri Hota						
Construction of Prototype	Debapriya Mukherjee						
Deployment, Delivery and Feedback	Mandira Pal						

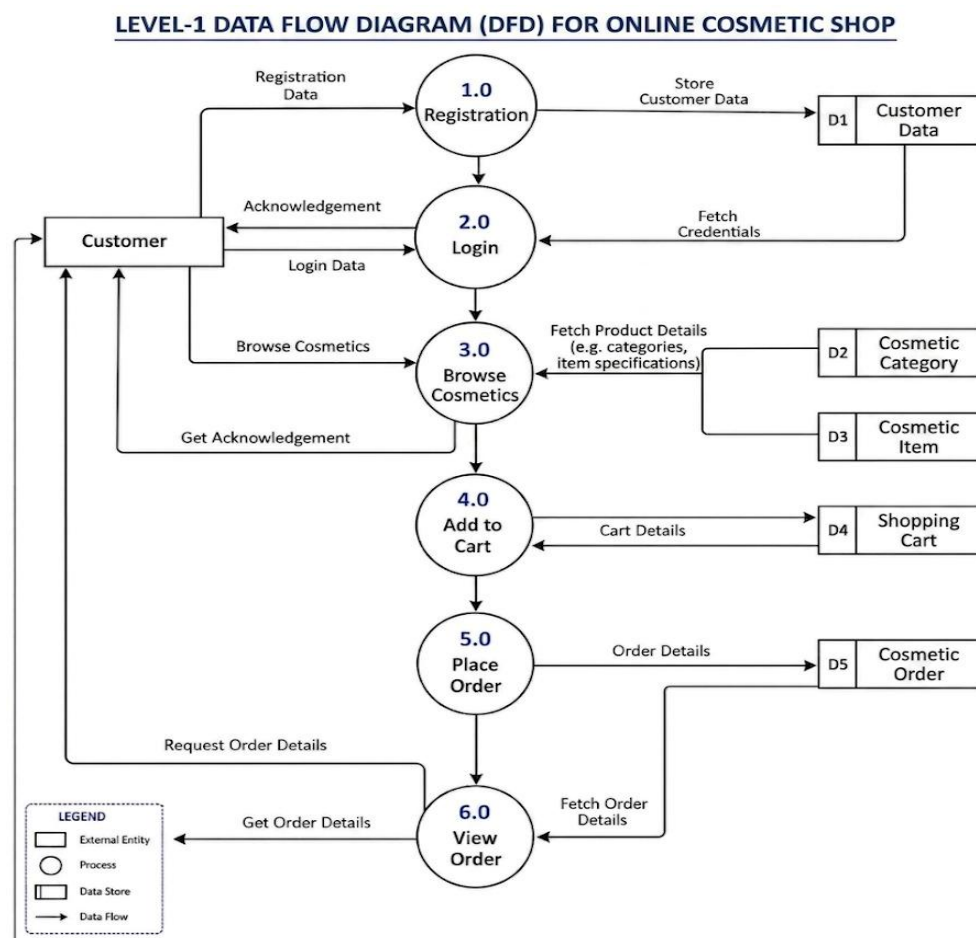
13. Software Engineering Paradigm Applied

Data flow diagram:

Level 0:

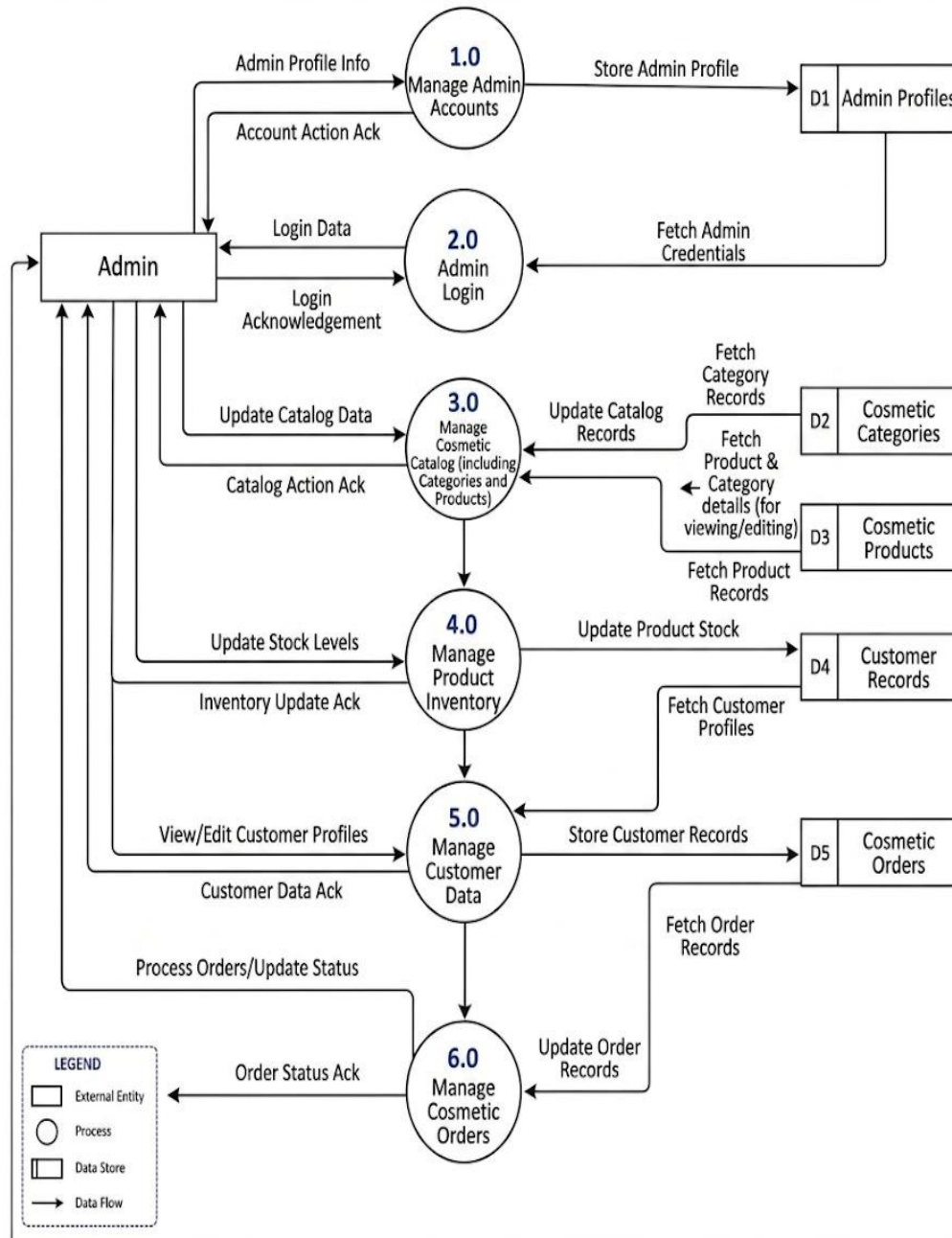


Level 1 for User :



Level 1 for Admin:

LEVEL-1 DATA FLOW DIAGRAM (DFD) FOR ONLINE COSMETIC SHOP - ADMIN



14. Schema/Database Design (project)

ItemTable :-

Attribute Name	Data type	Key	Description
ItemID	INT	PK	Unique user id
name	Varchar(100)	-	Item Name
description	TEXT	UNIQUE	Item description
price	DECIMAL(10,2)	-	Item price
image	VARCHAR(255)	-	Item image path

Category Table :-

Attribute	Data type	Key	Description
Cart_name	VARCHAR(100)	PK	Category name
about	TEXT	-	Category description
image	VARCHAR(255)	-	Category description

Order Table :-

Attributes	Data Type	Key	Description
OrderID	INT	PK	Unique Order Id
quantity	INT		Order Quantity
date_orderd	DATE	-	Order date
Payment_status	VARCHAR(30)	-	Payment Status
Payment_id	VARCHAR(50)	-	Payment Transition Id
Address	Varchar(255)	-	Delivery Address
mobile	VARCHAR(15)	FK	References continue (mobile)

Cart Item Table:-

Attribute	Data Type	Key	Description
product	INT	FK	Referances Item(ItemID)
Quantity	INT	-	Quantity of Item

Wishlist Table:

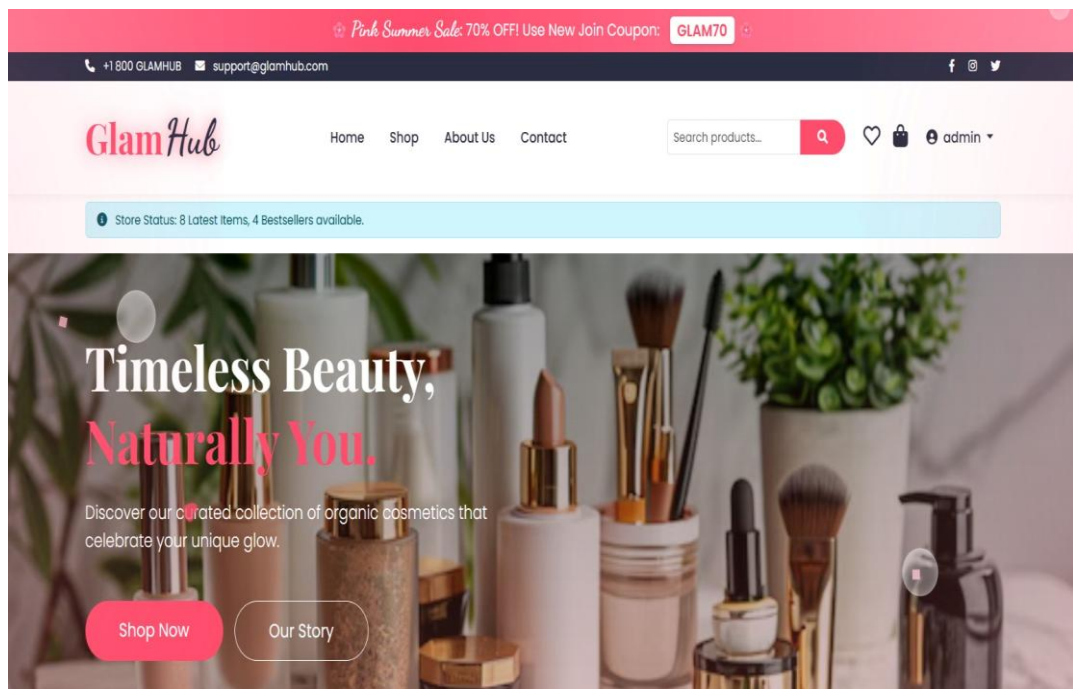
Attribute	Data Type	Key	Description
product	INT	FK	Referances Item(ItemID)
date	DATE	-	Wishlist creation

Customer Table:

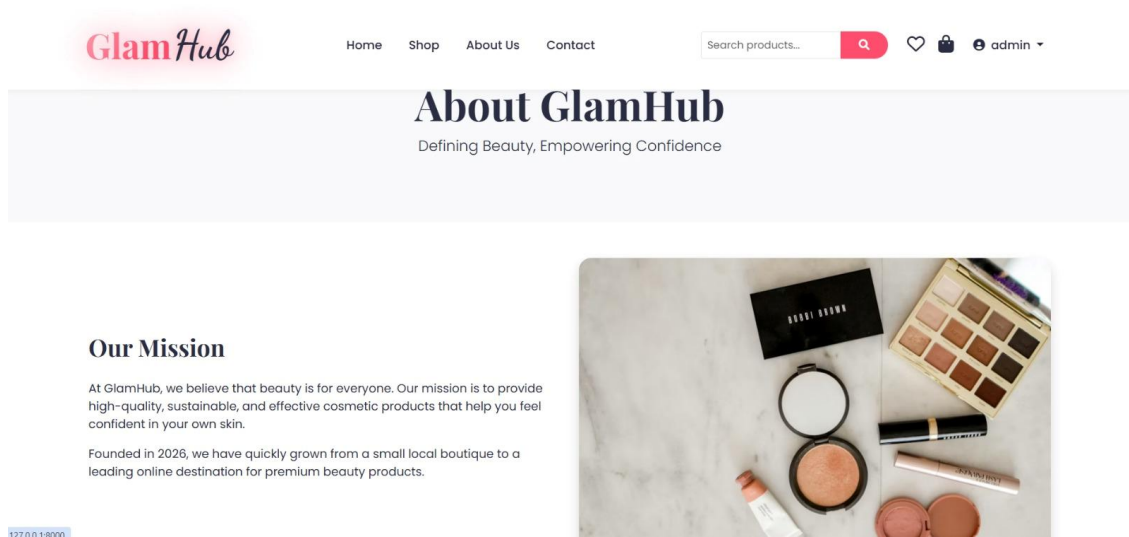
Attribute	Data Type	Key	Description
mobile	Varchar(15)	PK	Customer mobile number
name	Varchar(100)	-	Customer name
email	Varchar(100)	-	Customer email address

15. User Interface Design

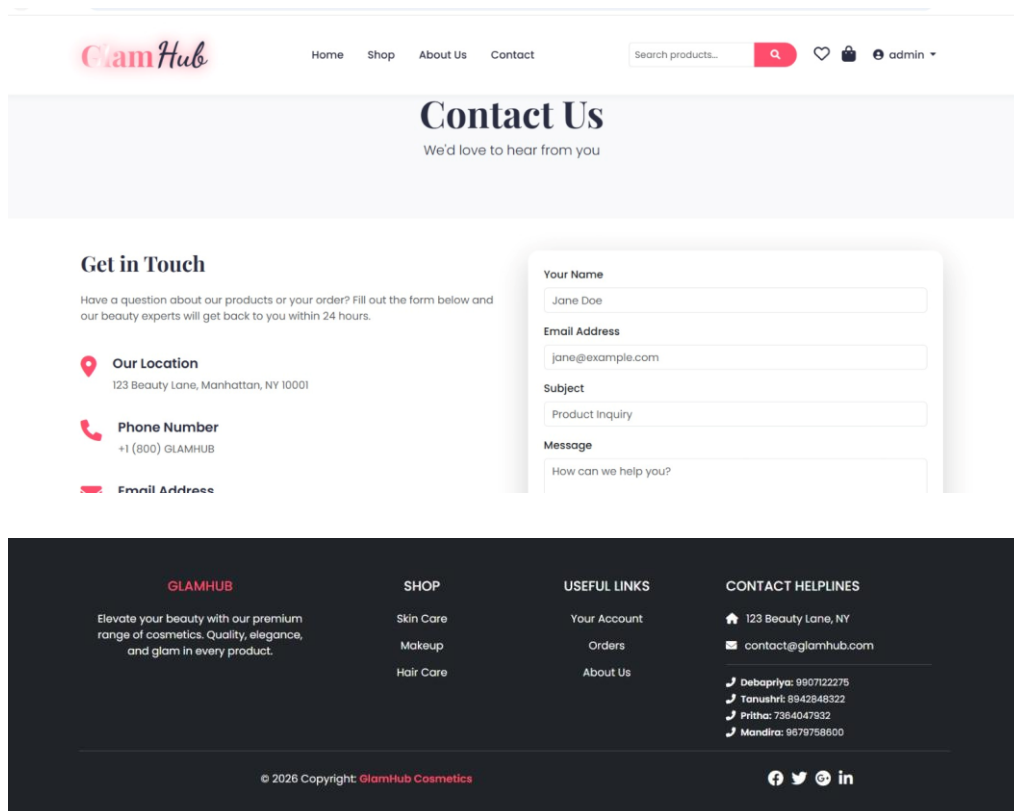
Home Page



About Us :



Contact Page :



Order Page :

[Home](#) [Shop](#) [About Us](#) [Contact](#)

Search products... 

   admin

Your Shopping Bag

[Home](#) / [Cart](#)

Order Summary

Subtotal **₹14494**

Summer Sale Discount (70%) **- ₹10145.80**

Shipping **Free**

Free Gift Unlocked!
 A glamorous surprise gift will be added to your delivery.

Final Total ₹4348.2

Delivery Address

WishList:

[Home](#) [Shop](#) [About Us](#) [Contact](#)

Search products... 

   admin

My Wishlist

[Home](#) / [Wishlist](#)



Radiance Serum
Skin Care



LipGloss
Makeup

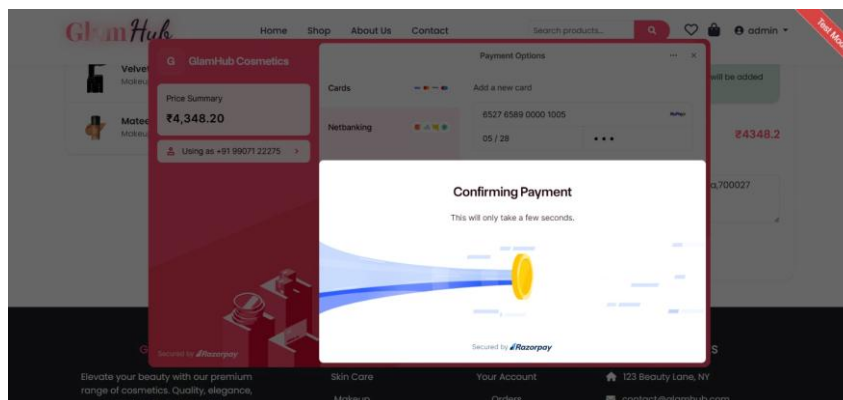
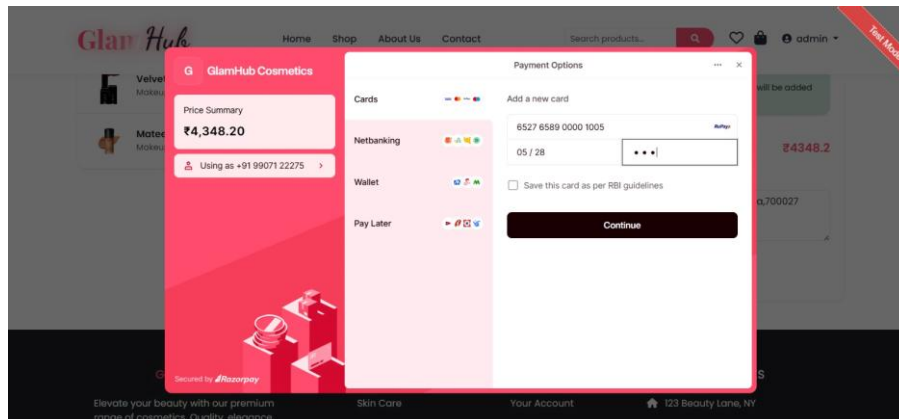


Jadore

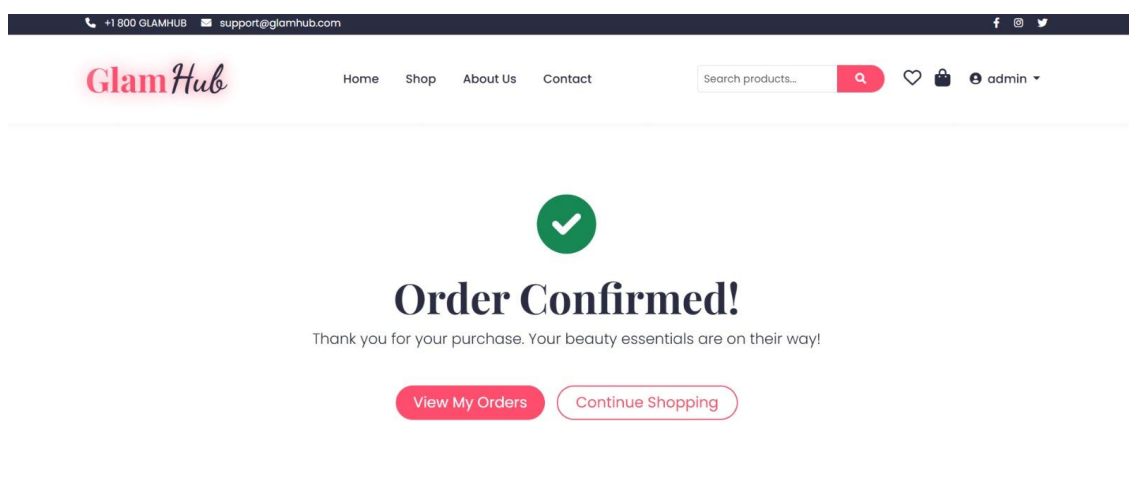


Eye Patches+spatula
Eye Care

Payment Page :



Order confirmation page:



Login into Admin panel :



[Home](#) [Shop](#) [About Us](#) [Contact](#)





[Login](#)

[Join Now](#)

Welcome Back

Login to your GlamHub account

Username

Password

Login

Don't have an account? [Sign Up](#)

Administration surface:

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

MYAPP

Cart items [+ Add](#)

Categorys [+ Add](#)

Items [+ Add](#)

Orders [+ Add](#)

Users [+ Add](#)

Action:

Go

 0 of 60 selected

☐	TITLE OF THE ITEM	CATEGORY	ITEM PRICE
☐	Makeup Brush Washer	Appliances	2618
☐	Sunscreen Powder	Sun Care	598
☐	Sun Expert Powder Matte	Sun Care	720
☐	Eye Patches + spatula	Eye Care	350
☐	Broad Spectrum SPF50	Sun Care	450
☐	Sensitive Skin Sunscreen	Sun Care	879
☐	White Oud	Fragrances	6459
☐	Essence of Elegance	Fragrances	9999
☐	Auto-Curl Wand	Appliances	5500
☐	Sonic Facial Cleanser	Appliances	2799
☐	Ceramic Hair Straightener	Appliances	4999
☐	Pro-Ion Hair Dryer	Appliances	3499
☐	Deep Nourish Shampoo	Hair Care	850
☐	Keratin Smooth Mask	Hair Care	1199
☐	Hyaluronic Plump Cream	Skin Care	1299
☐	Retinol Night Repair	Skin Care	1750
☐	Vitamin C Glow Serum	Skin Care	1599
☐	highlighter	Makeup	449
☐	Powder Blush	Makeup	449
☐	Coffee Golden Hour Glow combo	Body Care	1199

By category

All

Skin Care

Makeup

Hair Care

Fragrances

Body Care

Eye Care

Sun Care

Appliances

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups

+

Add

MTAPP

Cart Items

+

Add

Categories

+

Add

Items

+

Add

Orders

+

Add

Users

+

Add

Select order to change

Action:

Go

0 of 60 selected

☐	PRODUCT	USER	QUANTITY	PAYMENT STATUS	DATE ORDERED
☐	Himalayan Salt Scrub	admin	1	success	June 12, 2026, 2:36 p.m.
☐	Lavender Body Wash	admin	1	success	June 12, 2026, 2:36 p.m.
☐	Sensitive Skin Sunscreen	admin	1	success	June 12, 2026, 2:36 p.m.
☐	Anti-Aging Eye Cream	admin	1	success	June 12, 2026, 2:36 p.m.
☐	Ocean Breeze Cologne	admin	2	success	June 12, 2026, 2:36 p.m.
☐	Shampoo	admin	2	success	June 12, 2026, 2:36 p.m.
☐	Moisturizing Cream	admin	1	success	June 12, 2026, 2:36 p.m.
☐	Velvet Lipstick	admin	3	success	June 12, 2026, 2:26 p.m.
☐	Moisturizing Cream	admin	2	success	June 12, 2026, 2:26 p.m.
☐	Radiance Serum	admin	3	success	June 12, 2026, 2:26 p.m.
☐	Lavender Body Mist	debapriya.wcom@gmail.com	2	success	May 9, 2026, 3:32 p.m.
☐	Combopack of Shampoo and Conditioner	debapriya.wcom@gmail.com	1	success	May 9, 2026, 3:32 p.m.
☐	Cooling Eye Gel	debapriya.wcom@gmail.com	1	success	May 9, 2026, 3:32 p.m.
☐	Midnight Jasmine Perfume	debapriya.wcom@gmail.com	1	success	May 9, 2026, 3:32 p.m.
☐	Volume Mascara	debapriya.wcom@gmail.com	1	success	May 9, 2026, 3:32 p.m.
☐	Midnight Jasmine Perfume	debapriya.wcom@gmail.com	1	success	May 9, 2026, 9:58 a.m.
☐	Volume Mascara	debapriya.wcom@gmail.com	1	success	May 9, 2026, 9:58 a.m.
☐	Moisturizing Cream	debapriya.wcom@gmail.com	1	success	May 9, 2026, 9:58 a.m.

0 of 60 selected

ADD ORDER

Filter

Show counts

1 By payment status

All

success

1 By date ordered

Any date

Today

Next 7 days

This month

This year

Django administration

WELCOME ADMIN | VIEW SITE | CHANGE PASSWORD | LOG-OUT

Home

Myapp

Categories

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups

+

Add

MTAPP

Cart Items

+

Add

Categories

+

Add

Items

+

Add

Orders

+

Add

Users

+

Add

Select category to change

Action:

Go

0 of 8 selected

☐	NAME OF CATEGORY	ABOUT
☐	Appliances	High-performance beauty tools and electronic appliances for your daily routine.
☐	Sun Care	Protect your skin from harmful UV rays.
☐	Eye Care	Specialized treatments for the delicate eye area.
☐	Body Care	Pamper your body with our luxurious lotions and scrubs.
☐	Fragrances	Captivating scents for every occasion.
☐	Hair Care	Nourish and style your hair with our organic formulas.
☐	Makeup	Express your beauty with our professional makeup range.
☐	Skin Care	Premium products for your skin's health and glow.

8 categories

ADD CATEGORY

15. Coding

View.py code :

```

1  import razorpay
2  import re
3  from django.conf import settings
4  from django.http import JsonResponse
5  from django.shortcuts import render, redirect, get_object_or_404
6  from django.contrib import messages
7  from django.contrib.auth import authenticate, login, logout, update_session_auth_hash
8  from django.views.decorators.csrf import csrf_exempt
9  from django.contrib.auth.decorators import login_required
10 from .forms import CustomerForm, CartItemForm, MyChngForm, PwdChng
11 from .models import Item, Category, CartItem, Order, WishlistItem
12
13 def parse_price(price_str):
14     """Robustly extract the numeric price from a string like '7100/-' or '₹1,500'."""
15     try:
16         # Remove anything that isn't a digit or a decimal point
17         numeric_part = re.sub(r'[^0-9.]', '', price_str)
18         if '.' in numeric_part:
19             return float(numeric_part)
20         return int(numeric_part) if numeric_part else 0
21     except (ValueError, TypeError):
22         return 0
23
24 # Create your views here.
25
26 def home(request):
27     # Fetch all categories
28     categories = Category.objects.all()
29
30     # Fetch specific collections
31     # Using order_by('-id') ensures the latest items (the ones we just added) appear first
32     featured_items = Item.objects.all().order_by('-id')[:8]
33     bestsellers = Item.objects.filter(is_bestseller=True).order_by('-id')[:4]
34     new_arrivals = Item.objects.filter(is_new=True).order_by('-id')[:4]
35     top_rated = Item.objects.filter(is_top_rated=True).order_by('-id')[:4]

```

25

```

myapp ? views.py
26 def home(request):
27
28     # Placeholder images for categories
29     cat_images = [
30         "Skin Care": "https://images.unsplash.com/photo-1556228720-195a67268a01?q=80&auto=format&fit=crop",
31         "Hair Care": "https://images.unsplash.com/photo-1522138223662-42664741344c?q=80&auto=format&fit=crop",
32         "Nakeup": "https://images.unsplash.com/photo-1527799822344-42ad7c2827da?q=80&auto=format&fit=crop",
33         "Fragrances": "https://images.unsplash.com/photo-154164368914-70846636817q?q=80&auto=format&fit=crop",
34         "Body Care": "https://images.unsplash.com/photo-1552046122-03184de5e087?q=80&auto=format&fit=crop",
35         "Eye Care": "https://images.unsplash.com/photo-1512456015851-a9efb38a7967?q=80&auto=format&fit=crop",
36         "Sun Care": "https://images.unsplash.com/photo-150150869356-3c6b82a17d89?q=80&auto=format&fit=crop"
37     ]
38
39     for cat in categories:
40         if not cat_image:
41             cat_placeholder_image = cat_images.get(cat.cat_name, "https://images.unsplash.com/photo-152233789203-aabdfc54bc97?q=80&auto=format&fit=crop")
42
43     context = {
44         'featured_items': featured_items,
45         'categories': categories[0],
46         'bestsellers': bestsellers,
47         'new_arrivals': new_arrivals,
48         'top_rated': top_rated
49     }
50
51     return render(request, 'home.html', context)
52
53 def shop(request):
54     allcate = Category.objects.all()
55     allitem = Item.objects.all()
56     return render(request, 'shop.html', {'allitem': allitem, 'allcate': allcate})
57
58 def shopcate(request, id):
59     allcate = Category.objects.all()
60     allitem = Item.objects.filter(category=id)
61     return render(request, 'shop.html', {'allitem': allitem, 'allcate': allcate})
62
63
64
65
66
67
68
69
70
71
72 def about(request):
73     return render(request, 'about-us.html')
74
75 def contact(request):
76     return render(request, 'contact.html')
77
78 def custlog(request):
79     if request.method == "POST":
80         form = CustLogFrm(request=request, data=request.POST)
81         if form.is_valid():
82             uname = form.cleaned_data['username']
83             upass = form.cleaned_data['password']
84             user = authenticate(username=uname, password=upass)
85             if user is not None:
86                 login(request, user)
87                 messages.success(request, f'Welcome back, {user.first_name or user.username}!')
88                 return redirect('/')
89             else:
90                 form = CustLogFrm()
91     return render(request, 'login.html', {'form': form})
92
93 def custreg(request):
94     if request.method == "POST":
95         form = CustRegFrm(request.POST)
96         if form.is_valid():
97             try:
98                 form.save()
99                 messages.success(request, 'Your registration is successful. Please login.')
100                 return redirect('/login')
101             except Exception:
102                 messages.error(request, 'Your registration is not successful')
103     else:
104         form = CustRegFrm()
105
106     return render(request, 'custreg.html', {'form': form})
107
108 def custlogout(request):
109     logout(request)
110     return redirect('/login')
111
112 def add_to_cart(request, id):
113     if request.user.is_authenticated:
114         product = get_object_or_404(Item, id=id)
115         cart_item, created = CartItem.objects.get_or_create(product=product, user=request.user)
116         cart_item.quantity += 1
117         cart_item.save()
118         messages.success(request, f'{product.name} added to cart.')
119         return redirect('/shop')
120     else:
121         return redirect('/login')
122
123 def view_cart(request):
124     if request.user.is_authenticated:
125         cart_items = CartItem.objects.filter(user=request.user)
126         original_total = sum(parse_price(item.product.price) * item.quantity for item in cart_items)
127
128         # Apply 20% Discount (Pink Summer Sale)
129         discount_amount = original_total * 0.2
130         final_total = original_total - discount_amount
131
132     context = {
133         'cart_items': cart_items,
134         'original_total': original_total,
135         'discount_amount': discount_amount,
136         'total_price': round(final_total, 2)
137     }
138     return render(request, 'cart.html', context)
139     else:
140         return redirect('/login')

```

```

140
141 @login_required
142 def update_cart(request, item_id):
143     if request.method == 'POST':
144         quantity = int(request.POST.get('quantity', 1))
145         cart_item = get_object_or_404(CartItem, id=item_id, user=request.user)
146         if quantity > 0:
147             cart_item.quantity = quantity
148             cart_item.save()
149         else:
150             cart_item.delete()
151         messages.success(request, "Cart updated successfully.")
152         return redirect('/cart')
153
154 @login_required
155 def delete_cart_item(request, item_id):
156     if request.method == 'POST':
157         cart_item = get_object_or_404(CartItem, id=item_id, user=request.user)
158         cart_item.delete()
159         messages.success(request, "Item removed from cart.")
160         return redirect('/cart')
161
162 def account(request):
163     if request.user.is_authenticated:
164         if request.method == 'POST':
165             form = MyChngFrm(request.POST, instance=request.user)
166             if form.is_valid():
167                 try:
168                     form.save()
169                     messages.success(request, 'Profile updated successfully')
170                 except Exception:
171                     messages.error(request, 'Profile could not be updated')
172             else:
173                 form = MyChngFrm(instance=request.user)

```

```

myapp > views.py >...
172 def account(request):
173     form = MyChngFrm(instance=request.user)
174     return render(request, 'account.html', {'form': form})
175 else:
176     return redirect('/login')
177
178 @csrf_exempt
179 def initiate_payment(request):
180     if request.method == "POST":
181         try:
182             amount_str = request.POST.get("amount", "0")
183             # calculate amount in paise correctly, handling potential decimals
184             amount = int(round(parse_price(amount_str) * 100))
185             address = request.POST.get('address', '')
186             client = razorpay.Client(auth=(settings.RAZORPAY_API_KEY, settings.RAZORPAY_API_SECRET))
187
188             payment_data = {
189                 "amount": amount,
190                 "currency": "INR",
191                 "receipt": "order_receipt",
192                 "notes": {
193                     "email": request.user.email,
194                 },
195             }
196
197             order = client.order.create(data=payment_data)
198
199             response_data = {
200                 "id": order["id"],
201                 "amount": order["amount"],
202                 "currency": order["currency"],
203                 "key": settings.RAZORPAY_API_KEY,
204                 "name": "GlamHub Cosmetics",
205                 "description": "Payment for Your Order",
206                 "image": "https://yourwebsite.com/logo.png",
207             }

```

```

207
208
209     cart_items = CartItem.objects.filter(user=request.user)
210     for cart in cart_items:
211         Order.objects.create(user=request.user, product=cart.product, quantity=cart.quantity, payment_status='success', address=address)
212     CartItem.objects.filter(user=request.user).delete()
213
214     return JsonResponse(response_data)
215 except Exception as e:
216     return JsonResponse({"error": str(e)}, status=400)
217 return redirect('/payment-success')
218
219
220 def payment_success(request):
221     return render(request, "payment_success.html")
222
223 def my_orders(request):
224     orders = Order.objects.filter(user=request.user).order_by("-date_ordered")
225     return render(request, "my_orders.html", {"orders": orders})
226
227 def chngPwd(request):
228     if request.user.is_authenticated:
229         if request.method == 'POST':
230             form = PwdChng(user=request.user, data=request.POST)
231             if form.is_valid():
232                 user = form.save()
233                 update_session_auth_hash(request, user)
234                 messages.success(request, 'Password changed successfully')
235             else:
236                 messages.error(request, 'Please correct the errors below')
237         else:
238             form = PwdChng(user=request.user)
239         return render(request, 'chngPwd.html', {'form': form})
240     return redirect('/login')
241
242 def product_detail(request, id):

```

```

myapp > views.py > ...
241
242 def product_detail(request, id):
243     product = get_object_or_404(Item, id=id)
244     related_products = Item.objects.filter(category=product.category).exclude(id=id)[:4]
245     return render(request, 'product-details.html', {
246         'product': product,
247         'related_products': related_products
248     })
249
250 def search(request):
251     query = request.GET.get('q')
252     allcate = Category.objects.all()
253     if query:
254         allitem = Item.objects.filter(name__icontains=query)
255     else:
256         allitem = Item.objects.all()
257     return render(request, 'shop.html', {'allitem': allitem, 'allcate': allcate, 'query': query})
258
259 @login_required
260 def add_to_wishlist(request, id):
261     product = get_object_or_404(Item, id=id)
262     wishlist_item, created = WishlistItem.objects.get_or_create(product=product, user=request.user)
263     if created:
264         messages.success(request, f"{product.name} added to wishlist.")
265     else:
266         messages.info(request, f"{product.name} is already in your wishlist.")
267
268     # Redirect back to the page the user was on
269     next_url = request.GET.get('next', '/')
270     return redirect(next_url)
271
272 @login_required
273 def view_wishlist(request):
274     wishlist_items = WishlistItem.objects.filter(user=request.user)
275     return render(request, 'wishlist.html', {'wishlist_items': wishlist_items})
276

```

Url.py Code :

```

myapp > urls.py > ...
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home, name='hmpage'),
6     path('shop', views.shop, name='shoppage'),
7     path('shop/<int:id>', views.shopCate, name='shopcatpage'),
8     path('about', views.about, name='abtpage'),
9     path('contact', views.contact, name='cntctpage'),
10    path('login', views.custlog, name='loginpage'),
11    path('logout', views.custlogout, name='logoutpage'),
12    path('newcust', views.custReg, name='regpage'),
13    path('addtocart/<int:id>', views.add_to_cart, name='addtocart'),
14    path('cart', views.view_cart, name='crtpage'),
15    path('cart/update/<int:item_id>', views.update_cart, name='update_cart'),
16    path('cart/delete/<int:item_id>', views.delete_cart_item, name='delete_cart_item'),
17    path('account', views.account, name='account'),
18    path('initiate-payment/', views.initiate_payment, name='initiate_payment'),
19    path('payment-success/', views.payment_success, name='payment_success'),
20    path('my-orders/', views.my_orders, name='my_orders'),
21    path('chngpass', views.chngPwd, name='chngpass'),
22    path('product/<int:id>', views.product_detail, name='product_detail'),
23    path('search', views.search, name='search'),
24    path('wishlist', views.view_wishlist, name='view_wishlist'),
25    path('wishlist/add/<int:id>', views.add_to_wishlist, name='add_to_wishlist'),
26    path('wishlist/remove/<int:id>', views.remove_from_wishlist, name='remove_from_wishlist'),
27 ]
28

```

17. Testing

Team Interaction

The following describes the level of team interaction necessary to have a successful product.

- The Test Team will work closely with the Development Team to achieve a high quality design and user interface specifications based on customer requirements.

The Test Team is responsible for visualizing test cases and raising quality issues and concerns during meetings to address issues early enough in the development cycle.

- The Test Team will work closely with Development Team to determine whether or not the application meets standards for completeness. If an area is not acceptable for testing, the code complete date will be pushed out, giving the developers additional time to stabilize the area.
- Since the application interacts with a back-end system component, the Test Team will need to include a plan for integration testing. Integration testing must be executed successfully prior to system testing.

Test Objective

The objective our test plan is to find and report as many bugs as possible to improve the integrity of our program. Although exhaustive testing is not possible, we will exercise a broad range of tests to achieve our goal. We will be testing a Binary Search Tree Application utilizing a pre-order traversal format. There will be eight key functions used to manage our application: load, store, clear, search, insert, delete, list in ascending order, and list in descending order. Our user interface to utilize these functions is designed to be user-friendly and provide easy manipulation of the tree. The application will only be used as a demonstration tool, but we would like to ensure that it could be run from a variety of platforms with little impact on performance or usability.

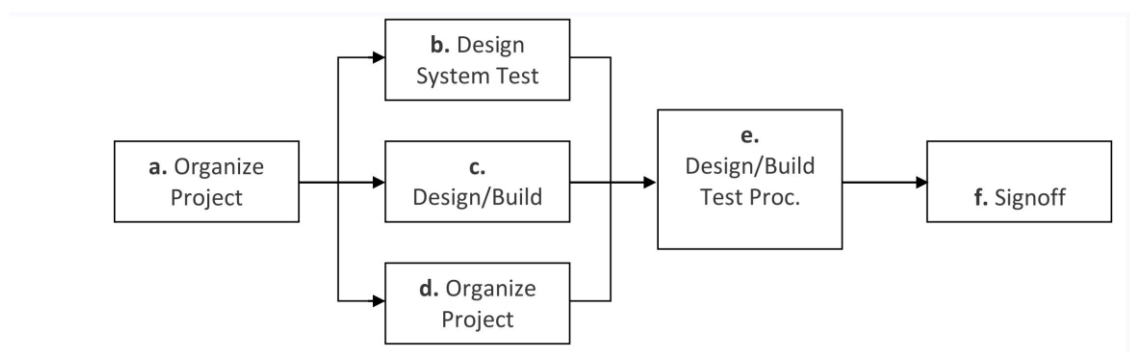
Process Overview

The following represents the overall flow of the testing process:

1. Identify the requirements to be tested. All test cases shall be derived using the current Program Specification.

2. Identify which particular test(s) will be used to test each module.
3. Review the test data and test cases to ensure that the unit has been thoroughly verified and that the test data and test cases are adequate to verify proper operation of the unit.
4. Identify the expected results for each test.
5. Document the test case configuration, test data, and expected results.
6. Perform the test(s).
7. Document the test data, test cases, and test configuration used during the testing process. This information shall be submitted via the Unit/System Test Report (STR).
8. Successful unit testing is required before the unit is eligible for component integration/system testing.
9. Unsuccessful testing requires a Bug Report Form to be generated. This document shall describe the test case, the problem encountered, its possible cause, and the sequence of events that led to the problem. It shall be used as a basis for later technical analysis.
10. Test documents and reports shall be submitted. Any specifications to be reviewed, revised, or updated shall be handled immediately.

Testing Process



The diagram above outlines the Test Process approach that will be followed.

a. **Organize Project** involves creating a System Test Plan, Schedule & Test Approach, and assigning responsibilities.

- b. Design/Build System Test** involves identifying Test Cycles, Test Cases, Entrance & Exit Criteria, Expected Results, etc. In general, test conditions/expected results will be identified by the Test Team in conjunction with the Development Team. The Test Team will then identify Test Cases and the Data required. The Test conditions are derived from the Program Specifications Document.
- c. Design/Build Test Procedures** includes setting up procedures such as Error Management systems and Status reporting.
- d. Build Test Environment** includes requesting/building hardware, software and data set-ups.
- e. Execute System Tests** – The tests identified in the Design/Build Test Procedures will be executed. All results will be documented and Bug Report Forms filled out and given to the Development Team as necessary.
- f. Signoff** - Signoff happens when all pre-defined exit criteria have been achieved.

Testing Strategy:

The following outlines the types of testing that will be done for unit, integration, and system testing. While it includes what will be tested, the specific use cases that determine how the testing is done will be detailed in the Test Design Document. The test cases that will be used for designing use cases is shown in Figure 2.1 and onwards.

Test Cases

Tested By:	Debapriya Mukherjee		
Test Type	Unit Testing		
Test Case Number	1		
Test Case Name	User Identification		
Test Case Description	The user should enter his/ her accurate userid and password so that he/she can able to go for the further options. The test case will check the application for the same since a user can only login with the correct userid , password.		
Item(s) to be tested			
1	Verification of the userid and password with the record in the database.		
Specifications			
Input		Expected Output/Result	
1) Correct User id and password		1) Successful login	
2) Incorrect Id or Password		2) Failure Message	

Unit Testing

Unit Testing is done at the source or code level for language-specific programming errors such as bad syntax, logic errors, or to test particular functions or code modules. The unit test cases shall be designed to test the validity of the programs correctness.

White Box Testing

In white box testing, the UI is bypassed. Inputs and outputs are tested directly at the code level and the results are compared against specifications. This form of testing ignores the function of the program under test and will focus only on its code and the structure of that code. Test case designers shall generate cases that not only cause each condition to take on all possible values at least once, but that cause each such condition to be executed at least once. To ensure this happens, we will be applying Branch Testing. Because the functionality of the program is relatively simple, this method will be feasible to apply.

Each function of the binary tree repository is executed independently; therefore, a program flow for each function has been derived from the code.

Black Box Testing

Black box testing typically involves running through every possible input to verify that it results in the right outputs using the software as an end-user would. We have decided to perform Equivalence Partitioning and Boundary Value Analysis testing on our application.

System Testing

The goals of system testing are to detect faults that can only be exposed by testing the entire integrated system or some major part of it. Generally, system testing is mainly concerned with areas such as performance, security, validation, load/stress, and configuration sensitivity. But in our case we will focus only on function validation and performance. And in both cases we will use the black-box method of testing

19. System Security measures (Implementation of security for the project developed)

- Only authorized users are allowed.
- Without signing in users are not allowed to go an intermediate page by typing an URL. For all such efforts, users will be redirected to the home page.

20. Database/Data security

- Database is present in remote machine.
- MySQL's default securities are applied.

21. Creation of User profiles and access rights

- The admin must create users manually
- The admin can create more admins

22. Cost Estimation of the Project along with Cost Estimation Model

Analogous estimate of effort or cost

Used for Early Estimate or Individual Activity Estimate

Sample example shown below is for two major deliverables of a software project. You use a previous project as a benchmark for analogous estimation. Using your experience you will estimate a multiplier.

Multipliers:

1. Prototyping: 0.75.
2. Testing: 0.5
3. Deployment: 0.5

Finally, if you want to convert to cost, you would use current rates for the resource.

WBS ID	Previous Similar Project Activity	Previous Effort	Current Project Estimate	Multiplier	Effort (Previous Effort * 0.75)	Cost (Rs. 500/hr.)
1	Prototyping	40 Work-Hours	Prototyping	0.75	30 Work-hours	Rs. 15000/-
2	Testing	20 Work-Hours	Testing	0.50	10 Work-Hours	Rs. 5000/-
Total					40 Work-Hours	Rs. 20000/-

Note: Effort is also called Size and unit of estimation is called either Work-Hour, person-hours.

23. Future scope and further enhancement of the Project

While the current application successfully handles core e-commerce functionalities like browsing, cart management, and discount calculations, the platform can be scaled further to improve user engagement and drive sales. The following features are proposed for future development:

- User-friendly online cosmetic shopping platform.
- Easy product search and browsing.
- Efficient cart and order management.
- Secure customer and admin authentication.
- Simple product and category management.
- Reliable database integration.
- Meets all project requirements successfully.
- Ready for future enhancements and scalability.

24. Bibliography

1. Roger S. Pressman. Software Engineering: A Practioner's Approach (Sixth Edition, International Edition). McGraw-Hill, 2005.
2. Ian Sommerville. Software Engineering (Seventh Edition). Addison-Wesley, 2004.
3. Frederick P. Brooks. The Mythical Man-Month: Essays on Software Engineering, Anniversary Edition. Addison-Wesley Pub Co; 1st edition (August 2, 1995).
4. Introducing HTML5 by Bruce Lawson, Remy Sharp.
5. HTML5 for Web Designer by Jeremy Keith.
6. HTML for World Wide Web (Visual Quick Start Guide) by Elizabeth Castro.
7. HTML5 Up And Running by Mark Pilgrim.
8. The Definitive Guide To HTML5 by Adam Freeman.
9. Pro HTML5 Programing: Powerful APIs for RcherInternet Aplication Development by Peter Lubbers.
10. Dynamic HTML: The Definitive Reference by Danny Godman.
11. CSS: Definitive Guide by Eric A. Mayer.
12. CSS Cook Book by Christopher Schmitt.
13. CSS: Missing Manual by David Sawyer McFarland.
14. CSS Mastery: Advance Web Standard Sollution by Andy Budd, Cameron Moll, Simon Collision.
15. CSS Anthology by Rachel Andrew.
16. Hancrafted CSS: More Bulletproof Web Design/ Bulletproof Essentials by Dan Caderholm, Ethan Marcotte.
17. Django for APIs by William S. Vincent.
18. Django for Professionals by William S. Vincent.
19. Two Scoops of Django by Daniel Roy Greenfeld and Audrey Roy Greenfeld.
20. Database System Concepts by Abraham Silverchatz, Henry F. Korth, S. Sudarshan.
21. SQL in 10 Minutes Sams Teach Yourself by Ben Forta.